

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

Entrega 6A

API y Catálogo de Inscripción

FASE 3 — Inscripciones (Inicio)

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Implementar una API Route POST que crea relaciones entre dos entidades (inscripción = estudiante + materia)
- Manejar el UNIQUE constraint para prevenir inscripciones duplicadas
- Crear una API Route GET que devuelve datos combinados de múltiples tablas (JOIN)
- Integrar el flujo de inscripción en el catálogo de materias existente
- Controlar el estado visual de un botón según el estado de la inscripción
- Usar el patrón de "estado local optimista" para mejorar la experiencia del usuario
- Comprender las transiciones de estado: inscrito → retirado (Entrega 6B)
- Aplicar toasts diferenciados para éxito, error y advertencia

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entregas 1A a 5C completadas y funcionando
- ✓ Tabla "inscripciones" creada en la base de datos (Entrega 1B)
- ✓ Al menos 5 materias en el sistema
- ✓ Al menos 1 usuario con rol "estudiante" para probar inscripciones
- ✓ La vista /estudiante/materias mostrando el catálogo con cards

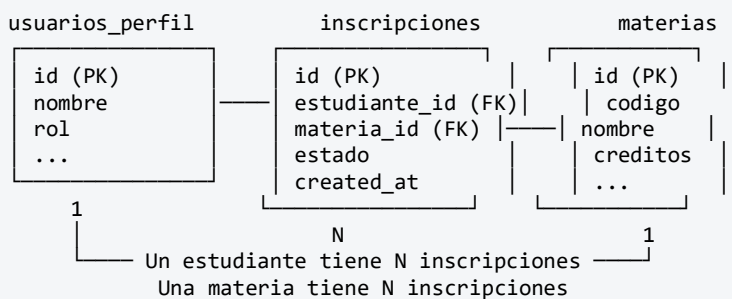
i Recordatorio del esquema: La tabla inscripciones tiene: id (UUID PK), estudiante_id (FK a usuarios_perfil), materia_id (FK a materias), estado (CHECK: 'inscrito', 'retirado'), created_at. Además tiene un UNIQUE(estudiante_id, materia_id) que impide inscripciones duplicadas.

Teoría: Inscripciones como Tabla de Relación

Relación Muchos-a-Muchos

Un estudiante puede inscribirse en muchas materias, y una materia puede tener muchos estudiantes inscritos. Esta relación muchos-a-muchos se implementa con una tabla intermedia: inscripciones.

Diagrama: Relación Muchos-a-Muchos



Estados de una Inscripción

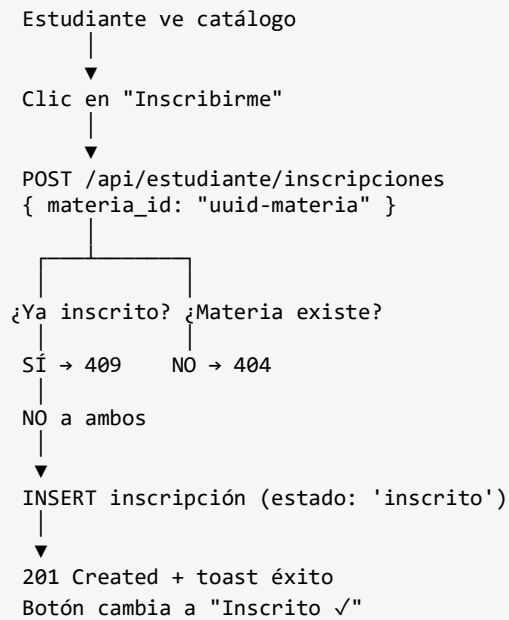
Cada inscripción tiene un campo "estado" con dos valores posibles:

Estado	Significado	Transición	Quién la ejecuta
inscrito	El estudiante está activamente inscrito	inscrito → retirado	El estudiante (Entrega 6B)
retirado	El estudiante se retiró de la materia	Estado final (no se reinscribe)	El estudiante (Entrega 6B)

⚠ Sin reinscripción: En nuestro MVP, una vez que un estudiante se retira de una materia, NO puede volver a inscribirse. El UNIQUE constraint permite solo un registro por par estudiante-materia. Esta es una simplificación pedagógica.

Flujo de Inscripción

Diagrama: Flujo de inscripción



☒ **UX inmediata:** Cuando el estudiante se inscribe exitosamente, el botón cambia de "Inscribirme" a "Inscrito" inmediatamente, sin necesidad de recargar la página.

Paso 1 — API Route: Inscribirse a una Materia (POST)

Este endpoint permite a un estudiante inscribirse en una materia. Valida la autenticación, el rol, la existencia de la materia y la duplicidad.

1.1 — Crear la estructura

Terminal

```
mkdir -p app/api/estudiante/inscripciones
```

1.2 — Crear la API Route

Crea el archivo `app/api/estudiante/inscripciones/route.ts`:

`app/api/estudiante/inscripciones/route.ts`

```
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

// ===== POST: Inscribirse a una materia =====
export async function POST(request: Request) {
  try {
    // 1. Verificar autenticación
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar que es estudiante
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "estudiante") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 3. Leer materia_id del body
    const body = await request.json();
    const { materia_id } = body;

    if (!materia_id) {
      return NextResponse.json(
        { error: "materia_id es requerido" },

```

```

        { status: 400 }
    );
}

// 4. Verificar que la materia existe
const { data: materia } = await supabase
    .from("materias")
    .select("id, codigo, nombre")
    .eq("id", materia_id)
    .single();

if (!materia) {
    return NextResponse.json(
        { error: "La materia no existe" },
        { status: 404 }
    );
}

// 5. Intentar insertar la inscripción
const { data, error } = await supabase
    .from("inscripciones")
    .insert({
        estudiante_id: user.id,
        materia_id: materia_id,
        estado: "inscrito",
    })
    .select(`
        id, estado, created_at,
        materia:materias!materia_id(id, codigo, nombre, creditos, semestre)
    `)
    .single();

if (error) {
    console.error("Error al inscribirse:", error);

    // UNIQUE violation: ya inscrito
    if (error.code === "23505") {
        return NextResponse.json(
            { error: "Ya estás inscrito en esta materia" },
            { status: 409 }
        );
    }

    return NextResponse.json(
        { error: "Error al procesar la inscripción" },
        { status: 500 }
    );
}

return NextResponse.json({ inscripcion: data }, { status: 201 });
} catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json(
        { error: "Error interno del servidor" },
        { status: 500 }
    );
}
}

```

1.3 — Puntos clave

Concepto	Detalle
Sin Service Role	El estudiante inserta en SU propia inscripción. RLS lo permite
estudiante_id = user.id	El servidor asigna el ID, no el cliente. Previene suplantación
Verificación de materia	Evitamos INSERT con un materia_id inválido (error FK)
Error 23505	El UNIQUE(estudiante_id, materia_id) impide duplicados
SELECT con JOIN	El response incluye datos de la materia para feedback inmediato
Status 201	Created: recurso nuevo creado exitosamente

✗ Seguridad: NUNCA permitas que el cliente envíe el estudiante_id en el body. Siempre usa user.id del token JWT. Si dejaras que el cliente enviara el ID, un estudiante podría inscribir a otro.

Paso 2 — API Route: Mis Inscripciones (GET)

Este endpoint devuelve todas las inscripciones del estudiante logueado, con los datos de cada materia.

2.1 — Agregar el handler GET

Abre `app/api/estudiante/inscripciones/route.ts` y agrega antes del POST:

```
app/api/estudiante/inscripciones/route.ts – agregar antes del POST
// ===== GET: Mis inscripciones =====
export async function GET() {
  try {
    // 1. Verificar autenticación
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar que es estudiante
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "estudiante") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 3. Obtener inscripciones con datos de materia y docente
    const { data: inscripciones, error } = await supabase
      .from("inscripciones")
      .select(`
        id, estado, created_at,
        materia:materias!materia_id(
          id, codigo, nombre, creditos, semestre,
          docente:usuarios_perfil!docente_id(nombre, apellido)
        )
      `)
      .eq("estudiante_id", user.id)
      .order("created_at", { ascending: false });

    if (error) {
      console.error("Error al obtener inscripciones:", error);
      return NextResponse.json(
        { error: "Error al obtener inscripciones" },
        { status: 500 }
      );
    }
  }
}
```

```

    return NextResponse.json({ inscripciones });
  } catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json(
      { error: "Error interno del servidor" },
      { status: 500 }
    );
  }
}

```

2.2 — SELECT con doble JOIN

Este endpoint usa una consulta con JOIN anidado: inscripciones → materias → usuarios_perfil (docente). Analicemos la sintaxis:

Estructura del SELECT anidado

```

.select(`
  id, estado, created_at,      // campos de inscripciones
  materia:materias!materia_id( // JOIN a materias
    id, codigo, nombre,      // campos de materias
    docente:usuarios_perfil!docente_id( // JOIN anidado
      nombre, apellido      // campos del docente
    )
  )
`)

```

El resultado JSON tiene esta estructura:

Ejemplo de respuesta

```

{
  "inscripciones": [
    {
      "id": "uuid-insc",
      "estado": "inscrito",
      "created_at": "2026-04-15T...",
      "materia": {
        "id": "uuid-mat",
        "codigo": "MAT-101",
        "nombre": "Cálculo I",
        "creditos": 6,
        "semestre": "2026-1",
        "docente": {
          "nombre": "María",
          "apellido": "López"
        }
      }
    }
  ]
}

```

☑ **JOINS anidados:** Supabase permite anidar relaciones FK indefinidamente. Aquí hacemos inscripciones → materias → docente en una sola consulta SQL, sin necesidad de múltiples fetch.

Paso 3 — Actualizar Catálogo con Botón de Inscripción

Ahora activaremos el botón "Inscribirme" que estaba deshabilitado en la Entrega 5C. El componente necesita saber en qué materias el estudiante ya está inscrito para mostrar el estado correcto.

3.1 — Reemplazar CatalogoMaterias completo

Reemplaza completamente el archivo `components/dashboard/estudiante/catalogo-materias.tsx`:

```
components/dashboard/estudiante/catalogo-materias.tsx – Parte 1/3
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import { Button } from "@components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { RefreshCw, ClipboardPlus, CheckCircle2, XCircle } from "lucide-react";
import MateriaCard from "@components/dashboard/materia-card";

interface Docente { nombre: string; apellido: string; }

interface Materia {
  id: string;
  codigo: string;
  nombre: string;
  creditos: number;
  semestre: string;
  docente: Docente | null;
}

interface Inscripcion {
  id: string;
  estado: "inscrito" | "retirado";
  materia: { id: string };
}

export default function CatalogoMaterias() {
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [inscripciones, setInscripciones] = useState<Inscripcion[]>([]);
  const [loading, setLoading] = useState(true);
  const [filtroSemestre, setFiltroSemestre] = useState<string>("todos");
  const [inscribiendo, setInscribiendo] = useState<string | null>(null);
```

i Nuevo estado: inscribiendo guarda el ID de la materia que se está procesando. Esto permite deshabilitar solo el botón correspondiente mientras se procesa la inscripción.

3.2 — Funciones de carga y acción

components/dashboard/estudiante/catalogo-materias.tsx – Parte 2/3

```
// Cargar materias e inscripciones en paralelo
const cargarDatos = async () => {
  setLoading(true);
  try {
    const [resMaterias, resInsc] = await Promise.all([
      fetch("/api/estudiante/materias"),
      fetch("/api/estudiante/inscripciones"),
    ]);

    const dataMaterias = await resMaterias.json();
    const dataInsc = await resInsc.json();

    if (resMaterias.ok) setMaterias(dataMaterias.materias);
    if (resInsc.ok) setInscripciones(dataInsc.inscripciones);
  } catch {
    toast.error("Error de conexión");
  } finally {
    setLoading(false);
  }
};

useEffect(() => { cargarDatos(); }, []);

// Verificar estado de inscripción de una materia
const getEstadoInscripcion = (materiaId: string) => {
  const insc = inscripciones.find(
    (i) => i.materia?.id === materiaId
  );
  if (!insc) return "no-inscrito";
  return insc.estado; // "inscrito" o "retirado"
};

// Inscribirse a una materia
const inscribirse = async (materiaId: string) => {
  setInscribiendo(materiaId);
  try {
    const res = await fetch("/api/estudiante/inscripciones", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ materia_id: materiaId }),
    });

    const data = await res.json();

    if (!res.ok) {
      toast.error(data.error || "Error al inscribirse");
      return;
    }

    toast.success(
      `Inscrito en ${data.inscripcion.materia.codigo} - ${data.inscripcion.materia.nombre}`
    );

    // Actualizar inscripciones localmente (optimista)
    setInscripciones((prev) => [...prev, data.inscripcion]);
  } catch {
    toast.error("Error de conexión");
  }
};
```

```
    } finally {  
      setInscribiendo(null);  
    }  
  }  
};  
  
const semestres = [...new Set(materias.map((m) => m.semestre))].sort();  
const materiasFiltradas = materias.filter(  
  (m) => filtroSemestre === "todos" || m.semestre === filtroSemestre  
);
```

☑ **Actualización optimista:** En lugar de recargar todo con `cargarDatos()`, agregamos la nueva inscripción al estado local con `setInscripciones(prev => [...prev, data])`. El botón cambia instantáneamente sin esperar al servidor.

3.3 — Renderizado actualizado

components/dashboard/estudiante/catalogo-materias.tsx – Parte 3/3

```
// Función que genera el botón según el estado
const renderBotonInscripcion = (materia: Materia) => {
  const estado = getEstadoInscripcion(materia.id);

  if (estado === "inscrito") {
    return (
      <Button variant="outline" size="sm" className="w-full" disabled>
        <CheckCircle2 className="mr-2 h-4 w-4 text-green-600" />
        Inscrito
      </Button>
    );
  }

  if (estado === "retirado") {
    return (
      <Button variant="outline" size="sm" className="w-full" disabled>
        <XCircle className="mr-2 h-4 w-4 text-red-500" />
        Retirado
      </Button>
    );
  }

  return (
    <Button
      size="sm"
      className="w-full"
      disabled={inscribiendo === materia.id}
      onClick={() => inscribirse(materia.id)}
    >
      <ClipboardPlus className="mr-2 h-4 w-4" />
      {inscribiendo === materia.id ? "Inscribiendo..." : "Inscribirme"}
    </Button>
  );
};

if (loading) {
  return (
    <div className="flex items-center justify-center py-12">
      <p className="text-muted-foreground">Cargando catálogo...</p>
    </div>
  );
}

return (
  <div className="space-y-4">
    {/* Barra de filtros */}
    <div className="flex items-center gap-4">
      <Select value={filtroSemestre} onChange={setFiltroSemestre}>
        <SelectTrigger className="w-[200px]">
          <SelectValue placeholder="Filtrar por semestre" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los semestres</SelectItem>
          {semestres.map((s) => (
            <SelectItem key={s} value={s}>{s}</SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
  </div>
);
```

```

<Button variant="outline" size="sm" onClick={cargarDatos}>
  <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
</Button>

<span className="text-sm text-muted-foreground ml-auto">
  {materiasFiltradas.length} materia(s) |{" "}
  {inscripciones.filter((i) => i.estado === "inscrito").length} inscrita(s)
</span>
</div>

{/* Grid de cards */}
{materiasFiltradas.length === 0 ? (
  <div className="flex items-center justify-center py-12">
    <p className="text-muted-foreground">No se encontraron materias</p>
  </div>
) : (
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
    {materiasFiltradas.map((m) => (
      <MateriaCard
        key={m.id}
        materia={m}
        mostrarDocente={true}
        accion={renderBotonInscripcion(m)}
      />
    ))}
  </div>
)}
</div>
);
}

```

3.4 — Estados visuales del botón

Estado inscripción	Botón	Variante	Icono	Habilitado
no-inscrito	"Inscribirme"	default (azul)	ClipboardPlus	Sí
inscribiendo...	"Inscribiendo..."	default (azul)	ClipboardPlus	No (disabled)
inscrito	"Inscrito"	outline (gris)	CheckCircle2 (verde)	No (disabled)
retirado	"Retirado"	outline (gris)	XCircle (rojo)	No (disabled)

i Contador actualizado: El texto del contador ahora muestra "5 materia(s) | 2 inscrita(s)" para dar feedback rápido de cuántas materias tiene el estudiante inscritas.

Paso 4 — Configurar RLS para Inscripciones

Para que el estudiante pueda insertar y leer sus propias inscripciones usando el cliente normal (sin Service Role), necesitamos configurar políticas RLS en Supabase.

4.1 — Ejecutar en Supabase Dashboard

Ve a Supabase Dashboard → SQL Editor y ejecuta:

SQL — Políticas RLS para inscripciones

```
-- 1. Habilitar RLS (si no está habilitado)
ALTER TABLE inscripciones ENABLE ROW LEVEL SECURITY;

-- 2. Estudiantes pueden ver SUS inscripciones
CREATE POLICY "estudiantes_ver_sus_inscripciones"
ON inscripciones
FOR SELECT
USING (auth.uid() = estudiante_id);

-- 3. Estudiantes pueden crear SUS inscripciones
CREATE POLICY "estudiantes_crear_inscripcion"
ON inscripciones
FOR INSERT
WITH CHECK (auth.uid() = estudiante_id);

-- 4. Estudiantes pueden actualizar SUS inscripciones (para retiro en 6B)
CREATE POLICY "estudiantes_actualizar_sus_inscripciones"
ON inscripciones
FOR UPDATE
USING (auth.uid() = estudiante_id)
WITH CHECK (auth.uid() = estudiante_id);
```

4.2 — Explicación de cada política

Política	Operación	Condición	Propósito
ver_sus_inscripciones	SELECT	auth.uid() = estudiante_id	Solo ve sus propias inscripciones
crear_inscripcion	INSERT	auth.uid() = estudiante_id	Solo puede inscribirse a sí mismo
actualizar_sus_inscripciones	UPDATE	auth.uid() = estudiante_id	Solo modifica sus inscripciones (para retiro)

⚠ Sin DELETE: No creamos política DELETE. Un estudiante no puede eliminar inscripciones, solo cambiar el estado a "retirado" (soft delete). Esto lo implementaremos en la Entrega 6B.

✓ **auth.uid():** La función `auth.uid()` de Supabase devuelve el UUID del usuario autenticado desde el token JWT. Es la base de toda seguridad RLS.

Paso 5 — Configurar RLS para Lectura de Materias

El catálogo del estudiante necesita leer la tabla materias. Si RLS está habilitado en materias, necesitamos una política de lectura. Si no está habilitado, este paso es opcional.

5.1 — Verificar y configurar RLS en materias

SQL — Políticas RLS para materias (lectura)

```
-- Habilitar RLS en materias (si no está habilitado)
ALTER TABLE materias ENABLE ROW LEVEL SECURITY;

-- Cualquier usuario autenticado puede VER materias
CREATE POLICY "usuarios_autenticados_ver_materias"
  ON materias
  FOR SELECT
  USING (auth.uid() IS NOT NULL);

-- También necesitamos lectura en usuarios_perfil para el JOIN del docente
ALTER TABLE usuarios_perfil ENABLE ROW LEVEL SECURITY;

CREATE POLICY "usuarios_autenticados_ver_perfiles"
  ON usuarios_perfil
  FOR SELECT
  USING (auth.uid() IS NOT NULL);
```

i Políticas amplias: Estas políticas permiten que cualquier usuario autenticado lea materias y perfiles. Esto es correcto: el catálogo de materias es público para todos los roles, y los nombres de docentes son información compartida.

⚠ Ya tenías Service Role: En las entregas de admin (4B, 5A, 5B) usamos SERVICE_ROLE_KEY que ignora RLS. Esas API Routes siguen funcionando sin cambios. Las políticas RLS solo afectan al cliente normal (PUBLISHABLE_KEY).

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo / Recurso	Acción	Descripción
app/api/estudiante/inscripciones/routes	Nuevo	GET: mis inscripciones + POST: inscribirme
components/dashboard/estudiante/catalogo-materias.tsx	Modificado (reemplazo)	Botón activo + estado de inscripción + actualización optimista
Supabase: RLS inscripciones	Nuevo (SQL)	3 políticas: SELECT, INSERT, UPDATE para estudiantes
Supabase: RLS materias	Nuevo (SQL)	Lectura para usuarios autenticados
Supabase: RLS usuarios_perfil	Nuevo (SQL)	Lectura para usuarios autenticados (JOIN docente)

Verificación Final

Ejecuta `npm run dev` y prueba con un usuario estudiante:

A. Catálogo con botones activos

- ✓ Iniciar sesión como estudiante → `/estudiante/materias`
- ✓ Todas las cards muestran botón "Inscribirme" azul y habilitado
- ✓ El contador muestra "X materia(s) | 0 inscrita(s)"

B. Inscripción exitosa

- ✓ Clic en "Inscribirme" en una materia
- ✓ El botón cambia a "Inscribiendo..." momentáneamente
- ✓ Aparece toast verde con el código y nombre de la materia
- ✓ El botón cambia a "Inscrito" con icono verde (CheckCircle2)
- ✓ El contador se actualiza a "X materia(s) | 1 inscrita(s)"

C. Inscripción duplicada

- ✓ Si recargas la página, la materia inscrita sigue mostrando "Inscrito"
- ✓ No existe forma de hacer doble clic (el botón se deshabilita)

D. Múltiples inscripciones

- ✓ Inscribirse en 2-3 materias diferentes funciona correctamente
- ✓ Cada botón se procesa independientemente (solo uno muestra "Inscribiendo...")
- ✓ El filtro de semestre no afecta el conteo de inscritas (es global)

E. Protección de API

- ✓ Llamar POST `/api/estudiante/inscripciones` sin auth devuelve 401
- ✓ Llamar como docente devuelve 403
- ✓ Enviar `materia_id` inválido devuelve 404

F. RLS funcional

- ✓ Las APIs de admin siguen funcionando (Service Role ignora RLS)
- ✓ La vista del docente /docente/materias sigue funcionando

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. Confirmación antes de inscribirse: Agrega un AlertDialog que muestre los detalles de la materia (código, nombre, créditos, docente) y pida confirmación antes de ejecutar la inscripción. Usa el componente AlertDialog que ya tienes instalado.
2. Límite de créditos: Agrega una validación en el servidor que impida inscribirse si el total de créditos del estudiante (sumando todas sus inscripciones activas) supera 30 créditos. Muestra un toast de advertencia con los créditos actuales y el límite.
3. Indicador visual en la Card: En lugar de solo cambiar el botón, agrega un borde verde (border-green-500) a las cards de materias en las que el estudiante ya está inscrito. Así se distinguen visualmente a primera vista.
4. Contador de créditos: Agrega un StatsCard encima del grid que muestre: Total de materias inscritas, Total de créditos inscritos (suma), y Créditos disponibles (30 - inscritos). Estos deben actualizarse dinámicamente con cada inscripción.

i Para entregar: Implementa al menos 2 de los 4 ejercicios. Inscríbete en al menos 4 materias y toma capturas del catálogo mostrando los diferentes estados de los botones.

Próximo: Entrega 6B

En la siguiente entrega completaremos el flujo de inscripciones:

- Página "Mis Inscripciones" del estudiante con tabla o cards de materias inscritas
- API Route PUT para cambiar estado de inscripción a "retirado"
- Botón "Retirar Inscripción" con AlertDialog de confirmación
- Badges de estado: inscrito (verde), retirado (rojo)
- Filtro por estado (inscrito / retirado / todos)

☒ **Preparación:** Asegúrate de tener al menos 4 inscripciones creadas con el catálogo de esta entrega. Las necesitarás para probar el retiro y los filtros en la Entrega 6B.

Fin de la Entrega 6A